Node.js - Node.js is kind of virtual environment that you need to install on your computer to be able to run JavaScript code or TypeScript code.

When JavaScript language was created, it was created only to be executable inside of the browser. And JavaScript is still the only one language that can be executed directly inside of the browser. And when Node.js was created, it allowed developer to run JavaScript outside of the browser and so build more complex and sophisticated systems.

And Since Playwright is built on JavaScript and on Typescript, to be honest, because JavaScript & Typescript is pretty much the same language with just the difference in types. And Since the Playwright was designed with the Typescript, we need to install Node.js to be able to run this package on your computer.

Node.js vs npm

	node	npm
What it is	JavaScript runtime	Package manager
What it does	Runs JS/TS code outside the browser	Installs and manages packages (TypeScript, Express, etc.)
Version check	`node -v`	`npm -v`
Maintained by	OpenJS Foundation	GitHub (Microsoft)
Example version	v22.x.x	10.x.x

Feature / Point	**Playwright**	**Cypress**
What it is	Modern end-to-end automation tool	JavaScript E2E testing tool
Primary language support	JS/TS + Java + Python + C#	Mainly JS/TS
Browser support	Chromium, Firefox, WebKit (Safari-like)	Strong in Chromium; limited Safari/WebKit compared to Playwright

Feature / Point	Playwright	Cypress
Architecture (how it runs)	Controls browsers via automation protocols (outside the browser)	Runs closer to the browser/app flow (different internal model)
Cross-browser testing	Very strong (built-in)	Possible, but comparatively limited
Multi-tab / new window handling	Strong support (tabs, popups, multiple contexts)	Traditionally limited; usually single-tab flow
Popup handling	Good support	Limited compared to Playwright
iFrame handling	Better support and smoother handling	Possible but often needs extra effort/plugins
Auto-waiting	Built-in auto-wait for actions and elements	Built-in retry/auto-wait behavior
Parallel execution	Strong built-in parallel execution support	Parallel execution possible, may depend on setup (Dashboard helps)
Debugging	Trace viewer, screenshots, videos, reports	Interactive runner + step-by-step debugging
CI/CD suitability	Very good for CI scaling and stability	Good, but setup style differs
Best suited for	Complex flows, cross-browser, multi-tab, enterprise CI	Fast local dev feedback, simpler single-tab web flows

When to choose

Situation	Better Choice	Why
Need true cross-browser testing (incl. WebKit/Safari-like)	Playwright	Built-in support for Chromium + Firefox + WebKit
App has popups / new tabs / multi-window flows	Playwright	Supports multiple tabs and contexts properly
Heavy iframe usage	Playwright	More reliable iframe support
Want best "interactive" local debugging experience	Cypress	Test runner UI is very developer-friendly
Team uses only JavaScript and wants quick setup	Cypress	Simple setup + strong dev experience
Want multi-language test support in future	Playwright	Supports Java/Python/C# too
Need strong CI parallel scaling out-of-the-box	Playwright	Parallel execution is easy with Playwright Test

Interview Script Questions & Answers: (Best for speaking in interviews)

Playwright vs Cypress

Q1) "Playwright vs Cypress — what's the difference?"

Script (30–40 sec):
"Playwright and Cypress both are used for web UI automation. The biggest difference is capability and architecture. Playwright supports cross-browser testing very well because it runs on Chromium, Firefox, and WebKit, and it handles complex flows like multi-tabs, popups, and iframes smoothly. Cypress is JavaScript-focused and has a very strong interactive runner for debugging, which is great for developers, but it can be limited for true multi-tab scenarios. For enterprise-level cross-browser coverage and CI scalability, I usually prefer Playwright."

Q2) "Which one is better for cross-browser testing?"

Script (15–25 sec):
"I would choose Playwright for cross-browser testing because it supports Chromium, Firefox, and WebKit out of the box. That gives wider coverage including Safari-like testing using WebKit. Cypress supports multiple browsers too, but Playwright's cross-browser support is more complete and straightforward."

Q3) "Your app opens a new tab during login/payment—what will you use?"

Script (15–25 sec):
"I'll use Playwright because it supports multiple tabs, popups, and browser contexts naturally. This is common in SSO logins and payment gateways. In Cypress, multi-tab handling is not as straightforward, so for these flows Playwright is the safer choice."

Q4) "How do you reduce flaky tests in these tools?"

Script (20–30 sec):
"Both Playwright and Cypress help reduce flakiness with auto-waiting and retries. In Playwright, I rely on built-in auto-wait and stable locators like getByRole/getByText, plus trace viewer for debugging. In Cypress, the runner is very helpful to debug step-by-step. The key is using good selectors, waiting for the right UI states, and avoiding hard waits."

Q5) "Which tool is better for CI/CD and parallel execution?"

Script (20–30 sec):
"Playwright is very strong for CI/CD because parallel execution is built into Playwright Test and it scales easily. Cypress also supports parallel execution, but it often depends on setup and sometimes Cypress Dashboard is used for better management. For quick scaling in CI, Playwright usually feels simpler."

Configuration of Development Environment (JavaScript + Playwright) — Full Setup (Download + Install + Run)

By the end, you will be able to **install everything**, **create a Playwright project**, **download browsers**, and **run + debug tests**.

---

1) What you must download (basic checklist)

You need these 3 things on your laptop:

1. **Node.js (LTS)**
2. **VS Code**
3. **Git** (recommended, not mandatory)

---

2) Download + Install Node.js (Most important)

Step-by-step

1. Open Google and search: **Node.js LTS download**
2. Open the official Node.js site and download **LTS** (not "Current")
3. Install it like normal software (Next → Next → Install)
4. Restart VS Code / restart PC if needed

Verify Node and npm
Open terminal (VS Code terminal or Command Prompt) and run:

- node -v
- npm -v

If both show versions, Node is installed correctly.

---

3) Download + Install VS Code

Step-by-step

1. Search: **VS Code download**
2. Download and install
3. Open VS Code

Recommended extension

- Open VS Code → Extensions → search **"Playwright Test for VS Code"** → Install

This helps you run tests and debug easily.

---

4) Download + Install Git (Recommended)

Why Git?
Because real projects use Git to store code in repos (GitHub/Azure DevOps)

Step-by-step

1. Search: **Git for Windows download**
2. Install it (default options are fine)
3. Verify:
    - git --version

---

5) Create your Playwright project (Proper way)

Now create your first Playwright folder.

Step-by-step

1. Create a folder anywhere, example:
   PlaywrightLearning
2. Open VS Code → File → Open Folder → select that folder
3. Open Terminal in VS Code (Terminal → New Terminal)

Now run this command (recommended):

- npm init playwright@latest

During setup it asks questions like:

- **Language?** (JavaScript or TypeScript)
- **Test folder name?** (usually tests)
- **Install browsers?** (Choose **Yes**)
- **Add GitHub Actions?** (Optional)

After this, Playwright setup is complete.

---

6) Browser download (Important to understand)

Playwright uses **its own browser binaries**:

- Chromium
- Firefox
- WebKit (Safari-like)

If for any reason browsers didn't install properly, run:

- npx playwright install

This downloads browsers again.

---

7) What files/folders get created (so you don't feel lost)

After installation you will see:

- package.json → your project dependencies and scripts
- playwright.config.* → main settings file (browser, timeout, reporter, etc.)
- tests/ → where your test files live
- playwright-report/ → HTML report
- test-results/ → screenshots/videos/traces (if enabled)

---

8) Running tests (your daily commands)

Run all tests:

- npx playwright test

Run and **see browser**:

- npx playwright test --headed

Run in **UI mode (best for learning)**:

- npx playwright test --ui

Open HTML report:

- npx playwright show-report

---

9) Debugging (very important)

When a test fails, use these:

Option 1: Debug mode

- npx playwright test --debug

Option 2: Pause inside test
Use:

- await page.pause();

This opens Playwright Inspector so you can inspect step-by-step.

Option 3: Trace viewer
If trace is enabled in config, Playwright provides replay of steps, which is very useful for debugging.

---

10) Common setup problems (quick fixes)

Problem: node command not found

- Fix: reinstall Node LTS and restart terminal

Problem: browsers not downloading

- Fix: npx playwright install
- Office network issue: try home network / VPN (if allowed)

Problem: tests not running

- Fix: ensure you are in correct folder (where package.json exists)

---

11) Interview-ready summary (2 lines)

- "I installed Node LTS + VS Code, initialized project with npm init playwright@latest, installed browsers, and run tests using Playwright Test runner. I use headed/debug modes and reports/traces for debugging."

Section- 2 – JavaScript Fundamentals

Module overview

- Node.js “Hello World” application
- Variables, Constants and Data Types
- Concatenation and Interpolation
- Objects and arrays
- Equality and Comparison operators
- Logical operators
- Conditional statement
- Loops
- Introduction to JavaScript Functions
- Introduction to JavaScript Classes and methods
- TypeScript vs JavaScript

“Hello World” in JavaScript

In this chapter, the goal is to confirm that the JavaScript setup works and a program can run successfully. “Hello World” is the smallest working program. If it runs properly, it means the editor, Node.js, and terminal setup are correct. Later, Playwright tests also run using the same Node environment, so this step is important.

What we cover here

We cover three basic things:

1. How we write JavaScript code in a file
2. How we run that file using Node.js
3. How we see output in the terminal using console.log()

Package.json

1. Overview

package.json is a small settings file that sits in our project folder. It describes our project: its name, its version, and some shortcut commands.

We create it with one command:

```bash
npm init -y
```

The `-y` means "yes to all default answers". That is why the instructor called it default. npm fills everything in for us.

Do we need it to run Hello World?

No. `node hello.js` works even without package.json. Courses still create it at the start because later we will install packages such as Playwright, and package.json keeps the list of those packages.

2. Core Concepts

After `npm init -y`, our package.json looks like this. The name comes from our folder name.

```json
{
  "name": "js-basics",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

What each line means, in simple words:

- **name:** our project's name.
- **version:** our project's version. We can leave it as `1.0.0`.
- **description, keywords, author:** optional information about the project. We can leave them empty.
- **main:** the main file of the project. We don't need it now.
- **scripts:** shortcut commands we can run with npm. This is the only part we will actually use for now.
- **license:** a legal license label. `ISC` is the default, and we can ignore it.

If our npm version is 11 or newer, we will also see `"type": "commonjs"` at the end. We can leave it as it is for now.

One rule to remember: package.json uses JSON format. Text must be inside double quotes, and there must be no comma after the last item.

3. Syntax and Examples

Step 1: create the project and a Hello World file.

```bash
mkdir js-basics
cd js-basics
npm init -y
```

```javascript
// hello.js
console.log("Hello World");
```

Step 2: run the file directly with Node.js.

```bash
node hello.js
```

Output:

```
Hello World
```

Step 3: add a shortcut script. We edit the `scripts` part of package.json:

```json
"scripts": {
  "start": "node hello.js",
  "test": "echo \"Error: no test specified\" && exit 1"
}
```

Now we can run the same file with:

```bash
npm start
```

Output:

```
> js-basics@1.0.0 start
> node hello.js

Hello World
```

The two lines starting with `>` are printed by npm. They show which script is running and the command behind it.

4. Playwright Relevance

At this stage we only need package.json to keep our project organised. When the course reaches Playwright, package.json becomes important. It will store the Playwright package and the commands we use to run our tests. We will add those details then.

5. Common Mistakes and Fixes

5.1 Running `npm test` and thinking something is broken

The default `test` script is only a placeholder. Running it prints:

```
Error: no test specified
```

This is normal. Nothing is wrong with our project.

5.2 Adding a comma after the last item

Wrong:

```json
"scripts": {
  "start": "node hello.js",
}
```

npm shows an `EJSONPARSE` error.

Correct:

```json
"scripts": {
  "start": "node hello.js"
}
```

5.3 Running a custom script without `run`

Only a few script names, such as `start` and `test`, work directly. Every other script needs `run`.

Wrong:

```bash
npm greet
```

This gives `Unknown command: "greet"`.

Correct:

```bash
npm run greet
```

6. Quick Practice

Task: in our `js-basics` folder, create `greet.js`:

```javascript
const course = 'JavaScript';
const day = 1;

console.log(`Day ${day}: Learning ${course} with Node.js`);
```

Then update the `scripts` part of package.json:

```json
"scripts": {
  "start": "node hello.js",
  "greet": "node greet.js",
  "test": "echo \"Error: no test specified\" && exit 1"
}
```

Run both scripts:

```bash
npm start
npm run greet
```

Expected output:

```
> js-basics@1.0.0 start
> node hello.js

Hello World

> js-basics@1.0.0 greet
> node greet.js

Day 1: Learning JavaScript with Node.js
```

1) Create the first JS file

Create a file named: hello.js

Write this inside:

```js
console.log("Hello World");
```

Meaning:

- console.log() prints something to the terminal.
- "Hello World" is a string (text).

---

2) Run it using Node.js

Open terminal inside the folder where hello.js exists and run:

bash Copy code

```
node hello.js
```

Expected output:

text Copy code

```
Hello World
```

If this output appears, the setup is working.

---

3) Common mistakes and fixes

Mistake: “node is not recognized”

Meaning: Node is not installed properly or PATH is not set.
Fix: Reinstall Node.js LTS and restart the terminal/PC.

Mistake: File not found / wrong folder

Meaning: Terminal is not opened in the folder where hello.js exists.
Fix: Navigate to the correct folder and run again.

Mistake: Typo in filename

Meaning: The command uses a different filename than the actual file.
Fix: Use the exact filename in the command.

---

4) Why this matters for Playwright

Playwright is executed in a similar way:

- Tests are JavaScript files
- They run using Node (npx playwright test)

So “Hello World” confirms the basic pipeline:
Write JS → Run with Node → See output

Quick mini practice

Try printing a custom message:

```js
console.log("Hello, Nitin!");
```

Variables, Constants and DataTypes

In JavaScript (and Playwright tests), we store values like URL, username, expected text, and flags (true/false).
For that we use variables and constants, and we must understand basic data types.

---

1) Variables and Constants

1.1 let (Variable)

Use **let** when the value can change later.

Example:

```js
let username = "nitin";
username = "rahul"; // allowed
```

1.2 const (Constant)

Use **const** when we should not reassign the value.

Example:

```js
const baseUrl = "https://example.com";
// baseUrl = "https://google.com"; // not allowed
```

Rule to follow:

- Use **const by default**
- Use **let only when value must change**

---

2) var (Old style)

var is older and can cause confusion due to scope behavior.
Modern JavaScript prefers **const** and **let**.

Example:

```js
var city = "Delhi"; // avoid in new code
```

---

3) Data Types (Common ones)

3.1 String (Text)

Example:

```js
const name = "Nitin";
const password = "abc@123";
```

3.2 Number

Example:

```js
const age = 30;
const price = 499.50;
```

3.3 Boolean (true/false)

Example:

```js
const isLoginSuccess = true;
const isVisible = false;
```

3.4 Undefined (Not assigned yet)

Example:

```js
let result;
console.log(result); // undefined
```

3.5 Null (Intentionally empty)

Example:

```js
const middleName = null;
```

3.6 Object (Key-value)

Example:

```js
const user = { name: "Nitin", role: "QA" };
```

3.7 Array (List of values)

Example:

```js
const browsers = ["chromium", "firefox", "webkit"];
```

---

4) Checking data type using typeof

Example:

```js
console.log(typeof "abc");       // string
console.log(typeof 10);          // number
console.log(typeof true);        // boolean
console.log(typeof undefined);   // undefined
console.log(typeof null);        // object (JavaScript quirk)
```

Note:

- typeof null shows "object" (this is a known JavaScript behavior)

---

5) Why this matters in Playwright

In automation we store:

- base URL (string)
- user credentials (object)
- list of browsers (array)

- status flags like visible/enabled (boolean)

Example:

```js
const baseUrl = "https://example.com";
const user = { email: "test@mail.com", password: "pass123" };
let loginStatus = false;
```

Quick Practice (Run this)

```js
const firstName = "Nitin";
let attempts = 1;
const isActive = true;

console.log(firstName);
console.log(attempts);
console.log(isActive);
console.log(typeof firstName, typeof attempts, typeof isActive);
```

Concatenation and Interpolation

In JavaScript (and Playwright tests), we often build messages and dynamic values like URLs, selectors, logs, and expected texts. For that, we join strings using **concatenation** or **interpolation**.

---

1) Concatenation (Using +)

Concatenation means joining strings using the + operator.

Example:

```js
const firstName = "Nitin";
const lastName = "Chaudhary";

const fullName = firstName + " " + lastName;
console.log(fullName); // Nitin Chaudhary
```

Common example:

```js
const status = "Passed";
console.log("Test Status: " + status);
```

Important note:

- If a number is added with a string, JavaScript converts the number to string.

Example:

```js
const attempts = 3;
console.log("Attempts: " + attempts); // Attempts: 3
```

---

2) Interpolation (Template Literals using backticks)

Interpolation means inserting variables inside a string using **template literals**.

Syntax:

- Use backticks:
- Put variable inside: ${variable}

Example:

```js
const firstName = "Nitin";
const lastName = "Chaudhary";

const fullName = `${firstName} ${lastName}`;
console.log(fullName); // Nitin Chaudhary
```

Common example:

```js
const status = "Passed";
console.log(`Test Status: ${status}`);
```

---

3) Why interpolation is preferred

Interpolation is preferred because:

- It is cleaner and more readable
- It avoids too many + signs
- It works nicely with multi-line strings

Multi-line example:

```js
const user = "Nitin";
const result = "PASS";

const message = `Execution Summary:
User: ${user}
Result: ${result}`;

console.log(message);
```

---

4) Real Playwright-style examples

Example 1 (Logging):

```js
const pageName = "Login Page";
console.log(`Navigating to: ${pageName}`);
```

Example 2 (Building a URL):

```js
const baseUrl = "https://example.com";
const path = "/login";

const loginUrl = `${baseUrl}${path}`;
console.log(loginUrl); // https://example.com/login
```

Example 3 (Expected message):

```js
const expected = "Login successful";
console.log(`Expected message is: ${expected}`);
```

---

5) Common mistakes

Mistake 1: Using quotes instead of backticks

```js
console.log("Hello ${name}"); // wrong, prints as-is
```

Correct:

```js
console.log(`Hello ${name}`);
```

Mistake 2: Missing ${ }

```js
console.log(`Hello name`); // prints "Hello name"
```

Correct:

```js
console.log(`Hello ${name}`);
```

Quick Practice (Run this)

```js
const product = "Gold Coin";
const qty = 2;
const price = 15000;

console.log("Product: " + product + ", Qty: " + qty);
console.log(`Total value = ${qty * price}`);
```

Objects & Arrays

In JavaScript (and Playwright automation), we often work with **structured data**.

- **Objects** are used to store related information in **key-value** form (like a user profile).
- **Arrays** are used to store a **list of values** (like multiple browsers, multiple users, multiple test data sets).

---

1) Objects (Key-Value Data)

1.1 What is an object?

An object stores data as **property: value** pairs.

Example:

```js
const user = {
  name: "Nitin",
  role: "QA",
  experience: 10
};
```

1.2 How to access object values

Using dot notation:

```js
console.log(user.name); // Nitin
console.log(user.role); // QA
```

Using bracket notation (useful when key is dynamic):

```js
console.log(user["experience"]); // 10
```

1.3 Updating object values

```js
user.role = "SDET";
console.log(user.role); // SDET
```

1.4 Adding a new property

```js
user.location = "India";
console.log(user.location); // India
```

1.5 Object with nested object (common in real data)

```js
const employee = {
  name: "Nitin",
  contact: {
    email: "nitin@test.com",
    phone: "9999999999"
  }
};

console.log(employee.contact.email); // nitin@test.com
```

---

2) Arrays (List of Values)

2.1 What is an array?

An array stores values in an ordered list.

Example:

```js
const browsers = ["chromium", "firefox", "webkit"];
```

2.2 Access array values using index

Index starts from 0:

```js
console.log(browsers[0]); // chromium
console.log(browsers[1]); // firefox
```

2.3 Common array operations

Add at the end:

```js
browsers.push("edge");
```

Remove last item:

```js
browsers.pop();
```

Get length:

```js
console.log(browsers.length);
```

---

3) Looping through an array (very common)

```js
const browsers = ["chromium", "firefox", "webkit"];

for (const b of browsers) {
  console.log(b);
}
```

---

4) Array of Objects (Most useful in automation)

This is used for test data sets.

Example:

```js
const users = [
  { email: "a@test.com", password: "pass1" },
  { email: "b@test.com", password: "pass2" }
];

console.log(users[0].email); // a@test.com
```

Loop and print:

```js
for (const u of users) {
  console.log(`Email: ${u.email}`);
}
```

---

5) Real Playwright-style examples

Example 1: Storing test data as object

```js
const testData = {
  baseUrl: "https://example.com",
  username: "testUser",
  password: "pass123"
};
```

Example 2: Storing multiple values as array

```js
const menuItems = ["Home", "Profile", "Settings"];
```

Example 3: Multiple users for data-driven testing

```js
const loginUsers = [
  { username: "user1", password: "p1" },
  { username: "user2", password: "p2" }
];
```

---

6) Common mistakes

Mistake 1: Using wrong key name

```js
console.log(user.nam); // undefined (typo)
```

Mistake 2: Wrong index

```js
console.log(browsers[5]); // undefined (out of range)
```

Quick Practice

```js
const student = {
  name: "Nitin",
  subjects: ["Math", "English", "Science"]
};

console.log(student.name);
console.log(student.subjects[1]); // English

student.subjects.push("History");
console.log(student.subjects);
```

Relational and Equality Operators

In JavaScript, operators help us compare values and make decisions. In Playwright automation, we use these comparisons all the time—for validating expected vs actual text, checking counts, verifying status codes, and deciding what action to take next.

---

1) Relational Operators (Compare “greater/less”)

Relational operators compare two values and return a boolean (true/false).

Operators and meaning

- > greater than
- < less than
- >= greater than or equal
- <= less than or equal

Equality operators compare “same or not same”.

- === strict equality (best for automation)

- !== strict not equal
- == loose equality (avoids type checking, can surprise us)
- != loose not equal

Examples

```js
// Strict equality (recommended)
console.log(10 === 10);     // true
console.log("10" === 10);   // false

// Loose equality (avoid in tests)
console.log("10" == 10);    // true (type conversion happened)
```

Realistic usage

const retryCount = 2;

console.log(retryCount <= 3); // true

String relational comparison (lexicographic / dictionary order)

console.log("b" > "a"); // true

console.log("10" > "2"); // false (string comparison, not numeric)

Important note
If both are strings, comparison is based on character order, not numeric value. For numeric comparisons, ensure values are numbers.

---

2) Equality Operators (Compare “same/not same”)

There are two types of equality checks in JavaScript:

A) Loose equality: == and !=

- Compares values after type conversion.

- Can cause unexpected results.

Examples

```
console.log(5 == "5");     // true
console.log(0 == false);   // true
console.log(null == undefined); // true
```

B) Strict equality: === and !== (Recommended)

- Compares value AND type.
- Most reliable for real projects and automation.

Examples

```
console.log(5 === "5");  // false
console.log(0 === false); // false
console.log(null === undefined); // false
console.log(5 !== "5");  // true
```

Rule to follow

- Use === and !== in automation unless there is a special reason.

---

3) Playwright-style examples (how it connects)

Example 1: Compare expected vs actual text

```
const actual = "Login successful";
const expected = "Login successful";

if (actual === expected) {
  console.log("Validation passed");
```

```
} else {
  console.log("Validation failed");
}
```

Example 2: Verify count is greater than zero

```
const records = 3;

if (records > 0) {
  console.log("Records found");
} else {
  console.log("No records found");
}
```

---

4) Common mistakes

Mistake 1: Using == and getting unexpected true

```
console.log("" == 0); // true (unexpected for beginners)
```

Mistake 2: Comparing numbers as strings

```
console.log("10" > "2"); // false (string compare)
```

Fix: Convert to number

```
console.log(Number("10") > Number("2")); // true
```

Quick Practice

```
const a = 10;
const b = "10";
```

```
console.log(a == b);   // ?

console.log(a === b);  // ?

console.log(a >= 5);   // ?

console.log(a < 5);    // ?
```

Chapter 11: Logical Operators (JavaScript)

Overview

Logical operators combine multiple conditions. In Playwright, we use them to check multiple validations together—like visible AND enabled, or login success OR error message.

---

1) AND operator &&

Both conditions must be true.

Examples

```
console.log(true && true);   // true

console.log(true && false);  // false
```

Automation-style example

```
const isVisible = true;

const isEnabled = true;


if (isVisible && isEnabled) {

  console.log("We can click the element");
```

```
}
```

---

2) OR operator ||

At least one condition must be true.

Examples

```
console.log(true || false);   // true
console.log(false || false);  // false
```

Automation-style example

```
const isChrome = false;
const isFirefox = true;

if (isChrome || isFirefox) {
  console.log("Run test on supported browser");
}
```

---

3) NOT operator !

Reverses boolean value.

Examples

```
console.log(!true);  // false
console.log(!false); // true
```

Automation-style example

```
const isErrorVisible = false;
```

```
if (!isErrorVisible) {
  console.log("No error shown, continue flow");
}
```

---

4) Short-circuit behavior (very important)

AND &&

- If first condition is false, second is not evaluated.

OR ||

- If first condition is true, second is not evaluated.

Example

```
const a = false;
const b = () => console.log("runs");

a && b(); // b() will not run
```

---

5) Common mistakes

Mistake: Using & instead of &&

- & is bitwise operator, not logical AND.

Correct

```
if (x > 0 && y > 0) { }
```

Quick Practice

```
const isLoggedIn = true;
```

```
const isAdmin = false;


console.log(isLoggedIn && isAdmin); // ?
console.log(isLoggedIn || isAdmin); // ?
console.log(!isAdmin); // ?
```

Chapter 12: Conditional Statements (JavaScript)

Overview

Conditional statements control program flow. In automation, conditions help handle dynamic cases like optional popups, different environments, or different user roles.

---

1) if statement

```
const age = 20;


if (age >= 18) {
  console.log("Allowed");
}
```

---

2) if...else

```
const isLoggedIn = false;


if (isLoggedIn) {
```

```
  console.log("Go to dashboard");
} else {
  console.log("Go to login");
}
```

---

3) else if ladder

```
const score = 75;

if (score >= 90) {
  console.log("Grade A");
} else if (score >= 70) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

---

4) Switch statement (useful for fixed options)

```
const browser = "chromium";

switch (browser) {
  case "chromium":
    console.log("Run on Chromium");
    break;
```

```
  case "firefox":
    console.log("Run on Firefox");
    break;
  default:
    console.log("Unsupported browser");
}
```

---

5) Ternary operator (short if-else)

```
const isVisible = true;
const msg = isVisible ? "Visible" : "Not Visible";
console.log(msg);
```

---

6) Playwright-style example (popup handling idea)

```
const isPopupVisible = false;

if (isPopupVisible) {
  console.log("Close popup");
} else {
  console.log("Continue");
}
```

Common mistakes

Mistake: Using = instead of ===

if (x = 5) { } // wrong (assignment)

Correct

if (x === 5) { }

Quick Practice

const value = 15;

if (value > 20) console.log("High");

else if (value >= 10) console.log("Medium");

else console.log("Low");

Chapter 13: Loops (JavaScript)

Overview

Loops repeat actions. In automation, loops help handle lists, multiple test data sets, multiple items in table, and repetitive validations.

---

1) for loop (most common)

```
for (let i = 1; i <= 3; i++) {
  console.log(i);
}
```

---

2) while loop

```
let i = 1;
while (i <= 3) {
  console.log(i);
  i++;
}
```

---

3) do...while loop

Runs at least once.

```
let i = 1;
do {
  console.log(i);
  i++;
} while (i <= 3);
```

---

4) for...of loop (best for arrays)

```
const browsers = ["chromium", "firefox", "webkit"];

for (const b of browsers) {
  console.log(b);
}
```

---

5) forEach (array method)

```
const items = ["A", "B", "C"];
```

```
items.forEach((item) => {
  console.log(item);
});
```

---

6) Break and continue

Break stops loop.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) break;
  console.log(i);
}
```

Continue skips one iteration.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i);
}
```

---

7) Playwright-style example (data driven idea)

```
const users = ["user1", "user2"];


for (const u of users) {
  console.log(`Testing login for ${u}`);
}
```

Common mistakes

Mistake: Infinite loop (forgetting increment)

```
let i = 1;
while (i <= 3) {
  console.log(i);
  // i++ missing => infinite loop
}
```

Quick Practice

```
const nums = [2, 4, 6, 8];
let sum = 0;

for (const n of nums) {
  sum = sum + n;
}
console.log(sum);
```

Chapter 14: Functions (JavaScript)

Overview

Functions are reusable blocks of code. In automation, functions reduce duplication: login steps, navigation, common validations, waits, and data formatting.

---

1) Function declaration

```
function greet(name) {
  return `Hello ${name}`;
}

console.log(greet("Nitin"));
```

---

2) Function expression

```
const add = function(a, b) {
  return a + b;
};

console.log(add(2, 3));
```

---

3) Arrow function (modern and common)

```
const multiply = (a, b) => a * b;
console.log(multiply(2, 5));
```

---

4) Default parameters

```
function printRole(role = "QA") {
  return role;
}
```

```
console.log(printRole());
console.log(printRole("SDET"));
```

---

5) Return vs no return

Return gives output to caller.

```
function square(n) {
  return n * n;
}
console.log(square(4));
```

No return means function ends without value.

```
function logMsg(msg) {
  console.log(msg);
}
logMsg("Hello");
```

---

6) Async functions (important for Playwright)

Playwright uses async/await.

Example format:

```
const doSomething = async () => {
  // await some async steps
};
```

---

7) Playwright-style example (reusable login)

```
const login = async (page, username, password) => {
  await page.fill("#username", username);
  await page.fill("#password", password);
  await page.click("#login");
};
```

Common mistakes

Mistake: forgetting return when needed

```
function add(a, b) {
  a + b; // no return
}
```

Correct:

```
function add(a, b) {
  return a + b;
}
```

Quick Practice

```
const isEven = (n) => n % 2 === 0;


console.log(isEven(4));
console.log(isEven(5));
```

Chapter 15: Classes and Methods (JavaScript)

Overview

Classes are used to organize code. In automation, classes are commonly used for Page Object Model (POM): each page becomes a class with locators and actions (methods).

---

1) Basic class and method

```
class User {
  constructor(name, role) {
    this.name = name;
    this.role = role;
  }

  getDetails() {
    return `${this.name} - ${this.role}`;
  }
}

const u1 = new User("Nitin", "QA");
console.log(u1.getDetails());
```

---

2) this keyword meaning

this refers to the current object created from the class.

Example:

```
class Car {
  constructor(brand) {
    this.brand = brand;
  }
}

const c = new Car("Honda");
console.log(c.brand);
```

---

3) Playwright POM style example (very common)

```
class LoginPage {
  constructor(page) {
    this.page = page;
    this.username = "#username";
    this.password = "#password";
    this.loginBtn = "#login";
  }

  async login(user, pass) {
    await this.page.fill(this.username, user);
    await this.page.fill(this.password, pass);
    await this.page.click(this.loginBtn);
```

```
  }
}
```

How it is used:

```
// const loginPage = new LoginPage(page);
// await loginPage.login("user", "pass");
```

Common mistakes

Mistake: forgetting new

```
const u = User("Nitin", "QA"); // wrong
```

Correct:

```
const u = new User("Nitin", "QA");
```

Quick Practice

```
class Calculator {
  add(a, b) { return a + b; }
}

const c = new Calculator();
console.log(c.add(2, 3));
```

Chapter 16: TypeScript vs JavaScript

Overview

JavaScript is dynamically typed. TypeScript adds type safety on top of JavaScript. Many Playwright projects use TypeScript because it improves maintainability and reduces runtime issues.

---

1) JavaScript example (no types enforced)

```
let value = "10";

value = 10; // allowed
```

---

2) TypeScript example (types enforced)

```
let value: number = 10;

// value = "10"; // not allowed
```

---

3) Benefits of TypeScript (in automation)

- Catch type errors during development (before running tests)
- Better autocomplete and IDE support
- Easier to maintain large test frameworks
- Clear function inputs/outputs

---

4) When to choose what

JavaScript is fine when:

- small learning projects
- quick scripts

TypeScript is better when:

- framework is large

- team work and long-term maintenance is needed

Quick Practice mindset

If project is long-term or team-based, TypeScript usually pays off.

If you want, next I can add for each chapter: **5 interview Q&A scripts** (speaking format) like we did for Playwright vs Cypress, but for these JavaScript topics.